COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 Data Structures and Algorithm	**SECTION:**	CPE21s1
DATE PERFORMED:	07/16/2026	**DATE SUBMITTED:**	07/16/2026
NAME:	Joshua Caleb P. Alvarez	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. __2.1_ **[ACTIVITY NAME]**Hands-on Activity 2.1 Arrays, Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

CODE

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and array.

```cpp
#include <iostream>
#include <string>

class Student{
    private:
        std::string studentName;
        int studentAge;

    public:
        //constructor
        Student(std::string newName = "John Doe", int newAge = 18){
            studentName = std::move(newName);
            studentAge = newAge;

            std::cout << "Constructor Called." << std::endl;
        }

        //destructor
        ~Student(){
            std::cout << "Destructor Called." << std::endl;
        }

        //Copy Constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called." << std::endl;
```

```
26              studentName = copyStudent.studentName;
27              studentAge = copyStudent.studentAge;
28          }
29
30          void printDetails(){
31              std::cout << this->studentName << " " << this->studentAge << std::endl;
32          }
33  };
34
35  int main(){
36      Student student1("Joshua", 19);
37      Student student2(student1);
38      Student student3;
39      student3 = student2;
40
41      return 0;
42  }
```

OUTPUT

```
PS C:\Alvarez CPE21S1> cd "c:\A
Constructor Called.
Copy Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Alvarez CPE21S1>
```

OBSERVATION

The Code works as follows:

- The library was firstly checked by the compiler
- The Class Declares two private data members: **`studentName`** and `studentAge`.
- The **Constructor**: Called whenever a new `Student` object is created.
- The message `"Constructor Called."` appears in your output each time an object is constructed.
- **Destructor**: Called when an object goes out of scope or is destroyed.
- Each `"Destructor Called."` line in your output corresponds to one object being destroyed.
- **Copy Constructor:** called when new object initialized from existing object.
- The Copy Constructor called appears in terminal which is the student 2 and 1.
- Void prints the student name and age and calls the constructor -> output which is constructor called.

Table 2-1. Initial Driver Program

PART B-**Run the code and show the output. Include the screenshot in table 2-1 followed by your observation**

CODE

```cpp
#include <iostream>
#include <string>

class Student{
    private:
        std::string studentName;
        int studentAge;

    public:
        //constructor
        Student(std::string newName = "John Doe", int newAge = 18){
            studentName = std::move(newName);
            studentAge = newAge;

            std::cout << "Constructor Called." << std::endl;
        }

        //destructor
        ~Student(){
            std::cout << "Destructor Called." << std::endl;
        }

        //Copy Constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called." << std::endl;
```

Run the code with the modified driver function and show the output. Include the screenshot in table 2-2 followed by your observation.

```
26              studentName = copyStudent.studentName;
27              studentAge = copyStudent.studentAge;
28          }
29
30          void printDetails(){
31              std::cout << this->studentName << " " << this->studentAge << std::endl;
32          }
33  };
34
35  int main(){
36      const size_t j = 5;
37
38      Student studentList[j] = {};
39      std::string nameList[j] = {"Carly","Freddy","Sam","Zack","Cody"};
40      int ageList[j]={15,16,18,19,16};
41      return 0;
42  }
```

OUTPUT

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Alvarez CPE21S1>
```

OBSERVATION

The Code works as follows:

- The `Student student1("Joshua", 19);` → **Constructor Called**
- The Student student 3 -> is the Constructor called
- `Student student3;` → **Constructor Called**
- `student3 = student2;` → **Assignment operator** (no message)
- When the program ends → **Three destructors** are called (for `student3`, `student2`, and `student1`).

Observations

The program demonstrates the use of constructors,deconstructors and copy constructors in C++ and also as my understanding between the three is constructors initializes objects when they are created While the copy constructor duplicates an existing object's data into a new one. Lastly the destructor is automatically called when the objects go out of scope, releasing resources.

The output confirms the correct sequence of objects creation and destruction.

Table 2-2. Modified Driver Program with Student Lists

Discuss what is done by loop A and loop B in table 2-3. Additionally, discuss the output and whether the functions are working as intended. If any corrections were made, further provide your modification and analysis in table 2-4

CODE

```cpp
#include <iostream>
#include <string>

class Student{
    private:
        std::string studentName;
        int studentAge;

    public:
        //constructor
        Student(std::string newName = "John Doe", int newAge = 18){
            studentName = std::move(newName);
            studentAge = newAge;

            std::cout << "Constructor Called." << std::endl;
        }

        //destructor
        ~Student(){
            std::cout << "Destructor Called." << std::endl;
        }

        //Copy Constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called." << std::endl;
```

```
26              studentName = copyStudent.studentName;
27              studentAge = copyStudent.studentAge;
28          }
29
30          void printDetails(){
31              std::cout << this->studentName << " " << this->studentAge << std::endl;
32          }
33  };
34
35  int main(){
36      const size_t j = 5;
37
38      Student studentList[j] = {};
39      std::string nameList[j] = {"Alejado","Aguila","Alvarez","Abello","Baltazar"};
40      int ageList[j]={21,19,19,19,16};
41
42
43      for(int i = 0; i < j; i++){//Loop A
44          Student* ptr = new Student(nameList[i], ageList[i]);
45          studentList[i] = *ptr;
46      }
47
48      for(int i = 0; i < j; i++){//Loop B
49          studentList[i].printDetails();
50
51      }
52      return 0;
53
54  }
55
```

LOOP A

```
for(int i = 0; i < j; i++){//Loop A
    Student* ptr = new Student(nameList[i], ageList[i]);
    studentList[i] = *ptr;
}
```

LOOP A OBSERVATION

- Each iteration dynamically creates a new `Student` object using `new`.
- The constructor runs, printing **"Constructor Called."** for each student.
- The object is then assigned into the `studentList` array.
- Since you didn't define an assignment operator, the compiler's default one is used.

LOOP B

```
for(int i = 0; i < j; i++){//Loop B
    studentList[i].printDetails();
```

LOOP B OBSERVATION

- Iterates through the `studentList` array.
- Calls `printDetails()` for each student, outputting their name and age.
- This confirms that the data from `nameList` and `ageList` was successfully copied into the `Student` objects.

OUTPUT

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Alejado 21
Aguila 19
Alvarez 19
Abello 19
Baltazar 16
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

OBSERVATION

The program demonstrates the fundamental principles of object-oriented programming in C++, including constructors, copy constructors, destructors, and assignment operations. When each `Student` object is created, the constructor initializes its attributes and prints a message confirming its execution. The copy constructor duplicates the data from an existing object, ensuring that each instance maintains its own values. The destructor is automatically invoked when objects go out of scope, signaling the end of their lifecycle. In **Loop A**, dynamic memory allocation is used to create new `Student` objects and assign them to an array, showing how objects can be managed in memory. In **Loop B**, the program iterates through the array and prints each student's details, verifying that the data was successfully stored and transferred. Overall, the code provides a clear demonstration of object creation, copying, and destruction processes, while also emphasizing the importance of proper memory management since dynamically allocated objects are not deleted, which may lead to memory leaks.

Table 2-3. Final Driver Program

MODIFICATION CODE

MODIFY.cpp > ...

```cpp
#include <iostream>
#include <string>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    // Constructor
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = newName;
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }

    // Copy Constructor
    Student(const Student &copyStudent) {
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
        std::cout << "Copy Constructor Called." << std::endl;
    }

    // Destructor
    ~Student() {
        std::cout << "Destructor Called." << std::endl;
```

```cpp
        std::cout << "Destructor Called." << std::endl;
    }

    void printDetails() {
        std::cout << "Name: " << studentName << ", Age: " << studentAge << std::endl;
    }
};

int main() {
    const int j = 5;
    std::string nameList[j] = {"Alejado", "Aguila", "Alvarez", "Abello", "Baltazar"};
    int ageList[j] = {21, 19, 19, 19, 16};

    Student studentList[j]; // array of Student objects

    // Loop A: Assign values to each student
    for (int i = 0; i < j; i++) {
        studentList[i] = Student(nameList[i], ageList[i]);
    }

    // Loop B: Display student details
    for (int i = 0; i < j; i++) {
        studentList[i].printDetails();
    }

    return 0;
}

```

OBSERVATION

The program illustrates the essential principles of object-oriented programming in C++, focusing on how constructors, copy constructors, and destructors manage the lifecycle of objects. When each `Student` object is created, the constructor initializes its attributes and prints a message confirming successful instantiation. The copy constructor duplicates the data of an existing object, ensuring that each instance maintains its own independent values. The destructor is automatically invoked when objects go out of scope, signaling the end of their existence and releasing any allocated resources.

In the main function, the program demonstrates both direct and copy-based object creation. Loop A dynamically generates student objects and assigns them to an array, showing how constructors are repeatedly called during object initialization. Loop B then iterates through the array to display each student's details, confirming that the data was correctly stored and transferred. The output sequence constructor calls followed by destructor calls validates the proper order of object creation and destruction.

Overall, the code provides a clear and practical example of how C++ handles object lifecycle management. It emphasizes the importance of constructors for initialization, copy constructors for duplication, and destructors for cleanup. The program also highlights the need for careful memory management, as dynamically allocated

objects should be properly deleted to prevent memory leaks. Through this demonstration, the activity helps me to understand how memory allocation works.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers
Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

CODE:

ILOC.cpp > Fruit

```cpp
#include <iostream>
#include <string>
using namespace std;


class Fruit {
private:
    string name;
    double price;
    int quantity;

public:
    // Constructor
    Fruit(string n = "", double p = 0, int q = 0) {
        name = n;
        price = p;
        quantity = q;
    }

    // Copy Constructor
    Fruit(const Fruit& other) {
        name = other.name;
        price = other.price;
        quantity = other.quantity;
    }
```

```
27      // Copy Assignment Operator
28      Fruit& operator=(const Fruit& other) {
29          if (this != &other) {
30              name = other.name;
31              price = other.price;
32              quantity = other.quantity;
33          }
34          return *this;
35      }
36
37      // Destructor
38      ~Fruit() {
39          cout << "Fruit Deleted: " << name << endl;
40      }
41
42      // Calculate total price
43      double calculateSum() const {
44          return price * quantity;
45      }
46
47      // Display information
48      void display() const {
49          cout << "Fruit: " << name
50               << " | Price: PHP " << price
```

```
50              << " | Price: PHP " << price
51              << " | Quantity: " << quantity
52              << " | Total: PHP " << calculateSum()
53              << endl;
54      }
55  };
56
57
58  class Vegetable {
59  private:
60      string name;
61      double price;
62      int quantity;
63
64  public:
65      // Constructor
66      Vegetable(string n = "", double p = 0, int q = 0) {
67          name = n;
68          price = p;
69          quantity = q;
70      }
71
72      // Copy Constructor
73      Vegetable(const Vegetable& other) {
74          name = other.name;
```

```
75          price = other.price;
76          quantity = other.quantity;
77      }
78
79      // Copy Assignment Operator
80      Vegetable& operator=(const Vegetable& other) {
81          if (this != &other) {
82              name = other.name;
83              price = other.price;
84              quantity = other.quantity;
85          }
86          return *this;
87      }
88
89      // Destructor
90      ~Vegetable() {
91          cout << "Vegetable Deleted: " << name << endl;
92      }
93
94      // Calculate total
95      double calculateSum() const {
96          return price * quantity;
97      }
```

```
100     void display() const {
101         cout << "Vegetable: " << name
102              << " | Price: PHP " << price
103              << " | Quantity: " << quantity
104              << " | Total: PHP " << calculateSum()
105              << endl;
106     }
107 };
108 
109 
110 double TotalSum(Fruit* fruits[], int fruitCount,
111                 Vegetable* vegetables[], int vegetableCount
112 {
113     double total = 0;
114 
115     for (int i = 0; i < fruitCount; i++)
116         total += fruits[i]->calculateSum();
117 
118     for (int i = 0; i < vegetableCount; i++)
119         total += vegetables[i]->calculateSum();
120 
121     return total;
122 }
123 
124 //==========================
125 // Main Function
```

```cpp
int main() {

    // Dynamic Memory Allocation
    Fruit* fruits[2];
    Vegetable* vegetables[2];

    fruits[0] = new Fruit("Apple", 10, 7);
    fruits[1] = new Fruit("Banana", 10, 8);

    vegetables[0] = new Vegetable("Broccoli", 60, 12);
    vegetables[1] = new Vegetable("Lettuce", 50, 10);

    cout << "===== Jenna's Grocery List =====" << endl;

    // Display fruits
    for (int i = 0; i < 2; i++)
        fruits[i]->display();

    // Display vegetables
    for (int i = 0; i < 2; i++)
        vegetables[i]->display();

    cout << endl;
```

```
150
151     // Display total amount
152     cout << "Total Amount: PHP "
153          << TotalSum(fruits, 2, vegetables, 2)
154          << endl;
155
156     cout << endl;
157
158     // Delete Lettuce
159     cout << "Deleting Lettuce..." << endl;
160     delete vegetables[1];
161     vegetables[1] = nullptr;
162
163     // Free remaining memory
164     delete fruits[0];
165     delete fruits[1];
166     delete vegetables[0];
167
168     return 0;
169 }
```

CODE ANALYSIS

The program demonstrates the use of **classes and object-oriented programming** in C++. It defines separate classes for fruits and vegetables, each containing private data members such as name, price, and quantity, along with constructors, copy constructors, copy assignment operators, destructors, and member functions to calculate and display item information.

The `calculateSum()` function computes the total price of each item by multiplying its price by its quantity, while the `display()` function prints the item's details in a readable format. The destructors display a message whenever an object is destroyed, demonstrating proper object cleanup and memory management.

Overall, the program shows how object-oriented programming can organize related data and functions into classes, making the code easier to understand, maintain, and reuse while demonstrating constructors, copy operations, destructors, and basic calculations.

OUTPUT

```
===== Jenna's Grocery List =====
Fruit: Apple | Price: PHP 10 | Quantity: 7 | Total: PHP 70
Fruit: Banana | Price: PHP 10 | Quantity: 8 | Total: PHP 80
Vegetable: Broccoli | Price: PHP 60 | Quantity: 12 | Total: PHP 720
Vegetable: Lettuce | Price: PHP 50 | Quantity: 10 | Total: PHP 500
```

```
Total Amount: PHP 1370

Deleting Lettuce...
Vegetable Deleted: Lettuce
Fruit Deleted: Apple
Fruit Deleted: Banana
Vegetable Deleted: Broccoli
```

```
Total Amount: PHP 1370
```

OUTPUT ANALYSIS

The program successfully displays Jenna's grocery list, showing the name, price, quantity, and total cost of each fruit and vegetable. It calculates the total amount of all items, which is **PHP 1370**. After deleting the lettuce object, the destructor is called, and the program displays messages indicating that the deleted objects have been removed from memory. The final total amount remains **PHP 1370** because the total was calculated before the deletion and was not recalculated afterward.

CONCLUSION

In this activity, I learned how to apply object-oriented programming concepts in C++ by creating classes and using constructors, copy constructors, copy assignment operators, and destructors. I also learned how member functions can perform calculations, display information, and organize data more efficiently. Through this program, I gained a better understanding of how objects are created, copied, and destroyed during program execution. Overall, this activity helped me improve my C++ programming skills and showed me the importance of writing organized, reusable, and easy-to-maintain code using object-oriented programming principles.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **COPILOT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **SUMMARY AND USE OF IDEA**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I use ai to help me observe the code**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.